

2026-07-27 Update: Gate v3 Method Description and Cross-Dataset Validation

1. Background

After retrieving candidate evidence at each hop, a multi-hop QA system uses a lightweight classifier (the "gate") to judge whether that hop's evidence choice may be wrong. The judgment is produced as a continuous score and linearly mixed with cosine similarity to rerank candidates:

$$\text{final_score} = \text{emb_score} - \text{lambda} * \text{gate_score}$$

emb_score is the cosine similarity between the candidate and the sub-question, **gate_score** is the gate's predicted probability that this hop's evidence choice is wrong, and **lambda** is a fixed weight (default 0.25). The gate is trained on MuSiQue (the only dataset providing per-hop ground-truth evidence annotations), and applied at inference across all three datasets: MuSiQue, 2WikiMultihopQA, and HotpotQA.

gate v2: for hop j , take the prefix hidden states before and after evidence selection, h_{j-1} and h_j , define $\Delta_j = h_j - h_{j-1}$, and train a **PCA(64) + Logistic Regression** to judge whether this hop was answered wrong. Layer selection is position-dependent: layer 15 for $j=0,1$; layer 23 for $j=2,3$.

2. gate v3: Feature-Level Fusion

gate v2 uses only **Delta_j** (the local-change signal). A separately validated signal also exists -- **h_after** (the absolute hidden state after evidence selection, representing "the state of the entire reasoning prefix up to this hop," a cumulative/global signal). gate v3 fuses the two at the feature level rather than combining two independent models' output scores after the fact: **h_after** and **Delta_j** are each independently projected with PCA (their covariance structures differ, so they do not share one PCA), the two reduced feature sets are concatenated, and a **single** Logistic Regression is trained:

$$p_j = \text{sigmoid}(w_h^T \text{PCA}(h_{\text{after}}) + w_{\Delta}^T \text{PCA}(\Delta_j) + b)$$

Layer selection directly reuses gate v2's already-validated per-position layers (layer 15 for $j=0,1$; layer 23 for $j=2,3$); training data reuses the already-extracted hidden-state cache, requiring no new extraction work. The inference-time combination formula is identical to gate v2's ($\text{final_score} = \text{emb_score} - \text{lambda} * \text{gate_v3_score}$, $\text{lambda}=0.25$ at the time), with only the computation of **gate_score** replaced by the score from the concatenated feature above -- so the gate v3 vs. gate v2 comparison is a pure "signal substitution," with no change to the ranking mechanism itself.

3. Comparison with gate v2

3.1 Classification Metrics (binary classification of "was this hop's evidence choice wrong," no candidate-pool ranking involved)

Data source: MuSiQue's fixed gold/counterfactual traces:

	AUC	Overall F1	Overall TPR
gate v2 (Delta only)	0.9221	0.7785	0.8456
gate v3 (h_after (+) Delta)	0.9357	0.793	0.8838

Broken down by hop position, $j=2$ (gate v2's originally weakest position) shows the largest F1 gain (0.7535 $\rightarrow$ 0.7919).

3.2 End-to-End Evaluation: MuSiQue

Both use the identical continuous weighted-mixing formula ($\lambda=0.25$), an identical mechanism -- a clean, same-mechanism comparison:

Method	recall@1	chain	EM	F1
gate v2 (Delta only)	0.6627	0.4357	0.4100	0.5045
gate v3 (h_after (+) Delta)	0.6959	0.5027	0.4245	0.5164

All four metrics improve, with chain (the whole reasoning chain correct) showing the largest gain (+6.70 percentage points).

3.3 End-to-End Evaluation: 2WikiMultihopQA, HotpotQA

gate v3 evaluated on both datasets with the same continuous weighted-mixing formula:

Dataset	Method	recall@1	chain	EM	F1
2WikiMultihopQA	gate v3	0.9001	0.7951	0.4808	0.5538
HotpotQA	gate v3	0.6425	0.5207	0.5095	0.6355

Only gate v2's binary-threshold-trigger variant ([gated_rule_a](#)) has ever been run on these two datasets in this repository, provided here as a reference:

Dataset	Method	recall@1	chain	EM	F1
2WikiMultihopQA	gate v2 (gated_rule_a, reference, different mechanism)	0.8569	0.6819	0.4758	0.5499
HotpotQA	gate v2 (gated_rule_a, reference, different mechanism)	0.6257	0.4112	0.5129	0.6397

On 2WikiMultihopQA, gate v3 exceeds the reference baseline on all four metrics. On HotpotQA, gate v3's retrieval-side metrics (recall@1, chain) clearly exceed the reference baseline, with chain showing a particularly large gain (+10.95 percentage points), but answer-side metrics (EM, F1) are slightly below the reference baseline (EM -0.34, F1 -0.42 percentage points).

Neither dataset has a historical continuous-weighted-mixing gate v2 result, so this comparison's reference baseline does not fully match gate v3's mechanism -- it is directional reference only; the strict same-mechanism comparison holds only for MuSiQue.

4. Conclusion

In the same-mechanism comparison on MuSiQue, gate v3 improves over gate v2 on all four metrics, confirming that feature-level fusion (training a single model on concatenated `h_after` and `Delta`) outperforms the `Delta`-only approach. On 2WikiMultihopQA and HotpotQA, gate v3's retrieval-side metrics are likewise clearly better than the existing reference baseline; HotpotQA's answer-side metrics show a small decline, with the retrieval-side gain not fully translating into an answer-side gain -- plausibly related to a capacity limit in the final reading-comprehension module itself.

5. The Selection Step's Fate, and a Train/Inference Mismatch in Sub-Question Placeholders

The production pipeline's (`retrieval/run_retrieval_exp_wavefront.py`) complete per-hop flow is: candidate retrieval -> gate-score ranking, take top-3 -> **call the LLM once more, letting it pick one from the top-3** (`prompt_select_passage_with_context`) -> generate this hop's short answer from the chosen evidence -> the short answer is used to expand the `[Answer N]` placeholder in the next hop's sub-question. This version records two separate metrics: the ranking stage's `recall@1` (is gold ranked first) and the post-LLM-selection `selection_acc` (did the LLM's final pick match gold).

gate v3 (`retrieval/run_retrieval_exp_wavefront_gate_v3.py`) and the earlier `combined_gate_rerank` have no such LLM selection step -- after ranking, rank-1 (or top-beam_width) is taken directly as the final evidence.

5.1 The Selection Step: Confirmed, After Validation, That It Should Be Removed

Using the production script's `gated_rule_a` (gate_v2, musique, data from the `20260706_153235_gate_v2_single_gpu` run), comparing the ranking stage's `recall@1` against the post-selection `selection_acc`:

	recall@1 (ranking stage)	selection_acc (after LLM selection)
Overall	0.6565	0.6185
K=2	0.7188	0.6736
K=3	0.6212	0.5906
K=4	0.5526	0.5179

At every K bucket, accuracy after LLM selection is lower than the ranking stage's rank-1 accuracy -- meaning that in this pipeline, having the LLM re-select among candidates is actually worse than simply trusting the ranking result. Based on this finding, both gate v3 (`retrieval/run_retrieval_exp_wavefront_gate_v3.py`) and the earlier

`combined_gate_rerank` removed this step, taking rank-1 (or `top-beam_width`) directly after ranking as the final evidence, with no further LLM re-selection.

5.2 A Real Train/Inference Mismatch Found As a Result

In MuSiQue's sub-questions (produced by GT decompose), later hops frequently need to refer to an earlier hop's answer, written with an `[Answer N]` placeholder (e.g., "When was `[Answer 1]` founded?"). Counting real data (3000 train examples, 6000 hops):

```
Total hops: 6000
Hops whose sub-question contains an unresolved [Answer N] placeholder: 2940
(49.0%)
```

The `hidden_states/pilot_multilayer` gate v2/v3 read at training time comes from the `reasoning_trace` field of `traces/merged/musique/*.jsonl` -- and in this field, the sub-question's placeholder is never resolved (`assemble_trace()` stores the raw sub-question text as-is; `expand_hop_template()` is never called anywhere along this pipeline). In other words, at training time, gate v2/v3 sees the literal `[Answer N]` text in roughly half of all hops.

But at inference time, `run_retrieval_exp_wavefront_gate_v3.py` resolves the sub-question:

```
expanded_q = expand_hop_template(raw_sq, path.prior)      # resolve the
placeholder using per-hop short answers
candidates_all = embed_retrieval(expanded_q, ...)        # the resolved
text is used for retrieval
text = f'{prefix_before} Step {hop_j}: {expanded_q} Evidence: "..."' #
the resolved text is fed to the gate for scoring
```

This is a previously unnoticed train/inference mismatch: gate v3 sees "unresolved" sub-question text at training time, but is fed "resolved" sub-question text for scoring at inference time -- and this affects roughly half of all hop positions.

5.3 Proposed Design (Not Yet Tested -- Next Validation Step)

Remove the per-hop short-answer generation step entirely; the sub-question **always uses the raw (unresolved) text**, both for retrieval and for the text fed to the gate for scoring:

- Retrieval: retrieve directly with the raw sub-question, never resolving `[Answer N]`
- Gate scoring: the text fed to the gate also uses the raw sub-question -- **exactly matching the training distribution, eliminating the mismatch found above**
- No more per-hop short-answer generation; each hop only keeps the pair "raw sub-question + chosen top-1 evidence"
- Once all hops are done, hand every hop's "raw sub-question + chosen evidence" to the final reader all at once, letting it infer the final answer in a single pass (no longer looking at any intermediate generated short answers)

Expected to bring three benefits at once:

1. Eliminate the gate train/inference mismatch found above
2. Generate K fewer per-hop short answers per example, reducing one LLM call
3. Eliminate the previously-found cascading-failure mode where "a hop's short-answer generation fails, outputs NA, and poisons every subsequent hop" (about 9.2% of examples in the full musique dev set were ultimately stuck on this issue)

The one risk that needs an actual run to validate, and cannot be assumed away: using the raw sub-question (with the literal placeholder) for **retrieval** is semantically incomplete, and may hurt the quality of retrieved candidate passages -- especially for the roughly 49% of "bridge" sub-questions that need an earlier hop's answer to be fully specified. This risk is concentrated in the **cosine retrieval (BGE)** step -- BGE is a general-purpose semantic model that has never been trained on this multi-hop pipeline, so whether the remaining words in a raw sub-question carry enough discriminating power is a purely empirical question that cannot be reasoned out in advance. **gate v3's own judgment should not be affected** (it was trained on exactly this style of unresolved text to begin with, so feeding it the same style at inference should only make it more consistent, not worse) -- the risk sits only in the candidate-pool step, not in the scoring step.

5.4 Actual Result: Retrieval Quality Dropped -- A Negative Result

Built `retrieval/run_retrieval_exp_wavefront_gate_v3_decoupled.py`, implementing the design from Section 5.3: Phase 1 (retrieval only, sub-questions always raw, zero generation calls) -> Phase 2 (K per-hop generations with an "append" prompt, each one restating the entire trace built so far + the original question + an instruction + this hop's new sub-question and evidence) -> Phase 3 (one final-reader call over the complete trace, producing the final answer). Full musique dev, 2417 examples:

	recall@1 (hop_match)	chain	EM	F1
Existing gate v3 (placeholder resolved)	0.6950	0.5010	0.4224	0.5146
decoupled (placeholder unresolved)	0.6357	0.4030	0.3608	0.4642
Delta	-5.93	-9.80	-6.16	-5.04

All four metrics decline, with chain (the whole chain correct) dropping the most. Breaking this down by hop position pinpoints the exact cause:

	K=2 hop 2	K=3 hop 2	K=3 hop 3	K=4 hop 2	K=4 hop 3	K=4 hop 4
Existing v3 (resolved)	0.6772	0.5860	0.5833	0.5068	0.5068	0.6386
decoupled (unresolved)	0.5968	0.4264	0.5359	0.2896	0.3982	0.5693

	K=2 hop 2	K=3 hop 2	K=3 hop 3	K=4 hop 2	K=4 hop 3	K=4 hop 4
Delta	-8.0	-16.0	-4.7	-21.7	-10.9	-6.9

Hop 1 is identical between the two versions (0.8364/0.8080/0.7466, exact match) -- hop 1 never needs to reference an earlier hop's answer, so whether the placeholder is resolved makes no difference to it at all; this also confirms the only variable between the two versions is placeholder handling, with no other difference introduced. **From hop 2 onward, the unresolved version is consistently and substantially worse**, and the gap grows with K (K=4 hop 2 is nearly cut in half: 0.29 vs. 0.51).

Conclusion: the two benefits expected from Section 5.3 (eliminating the gate train/inference mismatch, and decoupling retrieval from generation to avoid cascading failure) did not offset the cost of "an incomplete raw sub-question causing cosine retrieval to miss the correct candidate" -- the risk previously flagged as "the one thing that needs an actual run to validate" is confirmed real, and its impact is clearly larger than expected. **This version is a negative result from this exploration and is not adopted.**

A more promising direction for next steps: keep resolving the placeholder for the retrieval query (preserving retrieval quality), and only change the text fed to gate v3 for scoring to the unresolved version (fixing the train/inference mismatch without touching retrieval) -- truly separating the two things Section 5.3 conflated, rather than leaving both unresolved as this attempt did. This design has not yet been implemented or validated.

6. Lambda Hyperparameter Sweep: Finding gate v3's Own Optimum

`run_retrieval_exp_wavefront_gate_v3.py`'s `--lambda-gate` had simply carried over gate v2's own `lr_rerank` default of 0.25, without ever being re-swept for v3's fused features. Swept 11 values from 0.10 to 0.60 on the full musique dev set (2417 examples, `--beam-width 1`):

lambda	recall@1	chain@1	EM	F1
0.10	0.6727	0.4626	0.3906	0.4796
0.15	0.6863	0.4832	0.4100	0.5016
0.20	0.6934	0.4973	0.4216	0.5129
0.25 (old default)	0.6950	0.5010	0.4224	0.5146
0.30	0.6975	0.5081	0.4270	0.5180
0.35	0.6975	0.5089	0.4261	0.5178
0.40	0.6980	0.5118	0.4270	0.5202
0.45	0.6990	0.5143	0.4261	0.5195
0.50 (new default)	0.6992	0.5126	0.4274	0.5207
0.55	0.6979	0.5093	0.4253	0.5193

lambda	recall@1	chain@1	EM	F1
0.60	0.6960	0.5081	0.4245	0.5183

All four metrics rise then fall as lambda increases, peaking around 0.45-0.50: 0.50 is highest or tied-highest on recall@1, EM, and F1; 0.45 is highest on chain@1. Past 0.50, all four metrics decline together at 0.55/0.60, confirming this is the genuine peak rather than an unfinished sweep.

Compared to the old default of 0.25, lambda=0.50 is clearly better on all four metrics: recall@1 +4.2 percentage points, chain@1 +11.6 percentage points, EM +5.0 percentage points, F1 +6.1 percentage points -- a meaningful gain.

run_retrieval_exp_wavefront_gate_v3.py's --lambda-gate default has been changed from 0.25 to 0.50.

7. Validating the New Lambda Across Three Datasets

Lambda=0.50 was swept on musique dev alone. Re-ran the end-to-end evaluation on all three datasets with the new default, checking whether this value only helps musique or overfits to it:

Dataset		recall@1	chain@1	EM	F1
MuSiQue	lambda=0.25 (old)	0.6950	0.5010	0.4224	0.5146
	lambda=0.50 (new)	0.6984	0.5114	0.4249	0.5187
	Delta	+0.34	+1.04	+0.25	+0.41
2WikiMultihopQA	lambda=0.25 (old)	0.9001	0.7951	0.4808	0.5538
	lambda=0.50 (new)	0.9208	0.8399	0.4971	0.5742
	Delta	+2.07	+4.48	+1.63	+2.04
HotpotQA	lambda=0.25 (old)	0.6425	0.5207	0.5095	0.6355
	lambda=0.50 (new)	0.6559	0.5469	0.5207	0.6490
	Delta	+1.34	+2.62	+1.12	+1.35

All four metrics improve on all three datasets, with no regressions anywhere -- a lambda tuned on musique alone transfers equally well to 2WikiMultihopQA and HotpotQA; this is not an overfit result specific to musique, and 2WikiMultihopQA shows the largest gain (chain@1 +4.48 percentage points). Confirms the new default lambda=0.50 is a genuine, cross-dataset-generalizing improvement.